

Thinker AI V1 Architecture Spec

Synthesized from Founder Constitution (Extended), Constitution v1.0, and Founding Charter v1.0

June 2026

- [1. Purpose of This Document](#)
- [2. V1 Scope Statement](#)
 - [2.1 What V1 Is](#)
 - [2.2 What V1 Is Not](#)
 - [2.3 V1 Success Criteria](#)
- [3. Core Philosophy → Implementation Mapping](#)
- [4. System Architecture](#)
 - [4.1 Pipeline Overview](#)
 - [4.2 Thinker Core Responsibilities](#)
- [5. Agent Specifications](#)
 - [5.1 Intent Agent](#)
 - [5.2 Clarification Agent](#)
 - [5.3 Planner Agent](#)
 - [5.4 Research Agent](#)
 - [5.5 Builder Agent](#)
 - [5.6 Reviewer Agent](#)
 - [5.7 Critic Agent](#)
 - [5.8 Judge Agent](#)
 - [5.9 Consensus Agent](#)
- [6. Model & Cost Strategy](#)
 - [6.1 Model Tiers](#)
 - [6.2 Resource Protection Law in Practice](#)
 - [6.3 Direct Chat Path](#)
 - [6.4 When Consensus Actually Fires](#)
- [7. Data Flow & State](#)
 - [7.1 Pipeline State Object](#)
 - [7.2 Memory System](#)
- [8. Agent Fallibility & Error Containment](#)
 - [8.1 Why This Section Exists](#)
 - [8.2 Containment Rules](#)
 - [8.3 What This Does Not Solve](#)
- [9. Technology Stack \(V1 Recommendation\)](#)
- [10. Build Order \(Implementation Roadmap\)](#)
- [11. V1 Launch Checklist](#)
- [12. Explicitly Out of Scope for V1](#)
- [13. Open Decisions for Founder Review](#)

1. Purpose of This Document

The three founding constitutions describe Thinker AI's **vision and philosophy**: a multi-agent system that understands goals, plans work, debates solutions, and delivers verified outcomes. They do not specify **how to build it**.

This document translates that philosophy into a concrete, buildable Version 1 (V1) architecture. It defines

every agent, the data that flows between them, the technology choices, and the order of implementation — scoped deliberately small so V1 can actually ship.

Guiding constraint for this document: all nine agent roles named across the three constitutions are preserved, but each is implemented with the minimum logic needed to fulfill its role. Complexity is added in later versions, not V1.

2. V1 Scope Statement

2.1 What V1 Is

A web-based chat application where a user describes a goal (build an app, a website, a game, or a general task), and a fixed pipeline of nine lightweight agents — coordinated by Thinker Core — clarifies the request, plans it, builds a single deliverable, reviews it, and returns one final answer to the user.

2.2 What V1 Is Not

- Not a fully autonomous multi-day project system (that is a later phase in the long-term roadmap).
- Not a mobile app (V1 ships as a responsive web app; a mobile wrapper is a launch nice-to-have, not a blocker).
- Not running every request through full multi-model debate — multi-model consensus is invoked selectively (see Section 6.4), not on every single message, to protect cost.
- Not retaining credentials, passwords, or payment data in memory.

2.3 V1 Success Criteria

#	Criterion
1	User can chat normally and get direct answers without triggering the full agent pipeline
2	User can request a small project (app/website/game) and the system asks clarifying questions before building
3	All nine agent roles are present in code, even if some are simple rule-based stubs rather than full LLM calls
4	Every final deliverable passes through Reviewer before being shown to the user
5	Multi-model consensus works for at least one agent (Judge) end-to-end
6	Project memory persists within a session and reloads on return

3. Core Philosophy → Implementation Mapping

Constitution Principle	V1 Implementation
Understand first, plan second, execute third, verify always	Fixed pipeline order, enforced in code, not optional
Requirement Clarity Law — ask minimum questions before large work	Clarification Agent has a hard gate; Builder cannot run until requirements are marked complete
Token Economy / Resource Protection Law	Cheap single-model calls by default; multi-model consensus only for high-stakes decisions (Judge approval)
One Goal. One Team. One Final Answer.	Thinker Core merges all agent outputs into exactly one response object before returning to the user

No single model/agent/opinion is final — verified consensus is final	Judge Agent's approval is required before Thinker Core marks a deliverable "final"
--	--

4. System Architecture

4.1 Pipeline Overview

The pipeline runs in a fixed order, owned by Thinker Core:

1. **Thinker Core** receives the user message.
2. **Intent Agent** classifies it: chat, app request, website request, game request, or task request.
3. **Clarification Agent** checks if requirements are complete; if not, the pipeline halts and returns questions to the user.
4. **Planner Agent** breaks the clarified goal into ordered steps.
5. **Research Agent** optionally gathers external information for specific steps.
6. **Builder Agent** generates the actual deliverable.
7. **Reviewer Agent** checks logic, completeness, and basic security.
8. **Critic Agent** independently looks for weaknesses Reviewer may have missed.
9. **Judge Agent** scores the deliverable and decides approval, optionally invoking Consensus.
10. **Consensus Agent** runs a multi-model vote only when Judge's decision is borderline.
11. **Thinker Core** merges everything into one final response and returns it to the user.

4.2 Thinker Core Responsibilities

Thinker Core is not itself an LLM call — it is the **orchestration layer** (plain backend logic):

- Owns the pipeline state machine for each conversation
- Decides whether a message needs the full pipeline or just a direct chat reply
- Passes structured data between agents (see Section 7)
- Enforces the Clarification gate
- Merges Builder, Reviewer, Critic, and Judge outputs into one final response
- Writes to memory after each completed stage

5. Agent Specifications

Each agent is defined with its role, input, output, V1 implementation level, and model tier used.

5.1 Intent Agent

- **Role:** Classify the incoming message as chat, app request, website request, game request, or task request.
- **Input:** Raw user message plus the last five messages of context.
- **Output:** Intent type and a confidence score.
- **V1 Implementation Level:** Single LLM call with a classification prompt. No fine-tuning.
- **Model Tier:** Fast/cheap (Section 6.1).

5.2 Clarification Agent

- **Role:** Enforce the Requirement Clarity Law. Determine if enough information exists to proceed; if not, generate the minimum necessary questions.
- **Input:** Intent type plus the full conversation so far.
- **Output:** A completeness flag, a list of questions (if any), and the known requirements collected so far.

- **V1 Implementation Level:** LLM call against a checklist per intent type (for example, a website request needs category, audience, key features, and success criteria, drawn directly from the Website Creation Policy).
- **Model Tier:** Fast/cheap.
- **Gate Rule:** If requirements are not complete, Thinker Core returns the questions to the user and does not proceed to Planner. This is a hard stop, not a suggestion.

5.3 Planner Agent

- **Role:** Convert a clarified goal into an ordered list of concrete steps.
- **Input:** The known requirements object.
- **Output:** A list of steps, each with an id, description, and dependencies.
- **V1 Implementation Level:** Single LLM call producing a structured step list. No re-planning loop in V1 (re-planning on failure is a later feature).
- **Model Tier:** Mid-tier reasoning.

5.4 Research Agent

- **Role:** Gather external information when a step requires it (for example, “what does a modern fintech landing page look like”).
- **Input:** A single step from the Planner’s list, flagged as needing research.
- **Output:** Findings and sources.
- **V1 Implementation Level:** Optional — only invoked if Planner flags a step. Uses web search. Skipped entirely for purely generative steps, since most code and UI generation does not need it.
- **Model Tier:** Fast/cheap, with search tool access.

5.5 Builder Agent

- **Role:** Produce the actual deliverable — code, copy, configuration, or content — for each Planner step.
- **Input:** Step description, any research findings, and relevant project memory.
- **Output:** Artifact type, content, and any generated files.
- **V1 Implementation Level:** The most token-heavy agent. Runs once per step, sequentially, in V1 (parallel step execution is a later optimization).
- **Model Tier:** Strongest available coding/reasoning model.

5.6 Reviewer Agent

- **Role:** Run the quality control checklist: accuracy, logic, security, completeness, and user-goal alignment.
- **Input:** Builder’s output plus the original requirements.
- **Output:** A pass/fail flag and a list of issues with severity.
- **V1 Implementation Level:** Single LLM pass against the five-point checklist as explicit criteria. If it fails with high-severity issues, the request goes back to Builder once — capped at one retry in V1 to control cost.
- **Model Tier:** Mid-tier reasoning.

5.7 Critic Agent

- **Role:** Independently look for weaknesses that Reviewer’s checklist-based pass might miss — edge cases, unclear UX, fragile assumptions.
- **Input:** Builder’s output plus Reviewer’s findings.
- **Output:** A list of concerns with an overall severity rating.
- **V1 Implementation Level:** Single LLM call, deliberately given a different prompt persona than Reviewer (adversarial/skeptical framing) so it doesn’t just repeat Reviewer’s findings.
- **Model Tier:** Mid-tier reasoning, same tier as Reviewer but a different prompt.

5.8 Judge Agent

- **Role:** Score the deliverable on accuracy, feasibility, safety, cost, and completeness, then decide final approval.
- **Input:** Builder output, Reviewer result, and Critic concerns.
- **Output:** A score per criterion and an approval decision.
- **V1 Implementation Level:** If the combined score is clearly high or clearly low, Judge decides alone. If borderline (within a defined threshold of the pass line), Judge invokes the Consensus Agent instead of deciding unilaterally.
- **Model Tier:** Strongest available reasoning model.

5.9 Consensus Agent

- **Role:** Implement the debate-system / multi-model consensus described across all three constitutions, used only when Judge needs a tie-breaker.
- **Input:** The same question Judge was uncertain about.
- **Output:** Each model's vote and reasoning, plus a final verdict.
- **V1 Implementation Level:** Sends the identical question to three different models — one from each of three providers — collects their verdicts, and takes the majority vote. Ties default to the more conservative outcome (reject or ask the user).
- **Model Tier:** Three different providers, called in parallel.

6. Model & Cost Strategy

6.1 Model Tiers

Tier	Used By	Purpose
Fast/cheap	Intent, Clarification, Research	High-frequency, low-complexity decisions
Mid-tier reasoning	Planner, Reviewer, Critic	Structured reasoning, moderate complexity
Strongest available	Builder, Judge	Highest-stakes generation and final scoring
Multi-model (3 providers)	Consensus	Only invoked on borderline Judge decisions

6.2 Resource Protection Law in Practice

- Every pipeline run is logged with token cost per agent.
- The Clarification gate prevents Builder, the most expensive agent, from ever running against an unclear request.
- The Consensus Agent, roughly three times the cost of a single call, is invoked only when Judge's confidence is borderline — estimated at well under 20% of all requests.
- The Reviewer retry loop is capped at one retry to prevent runaway cost on a stuck request.

6.3 Direct Chat Path

Not every message needs the full nine-agent pipeline. If Intent Agent classifies a message as ordinary chat — a general question, a follow-up, or a clarification reply that doesn't start a new project — Thinker Core skips straight to a single direct-answer call and returns. The full pipeline is reserved for app, website, game, and task requests.

6.4 When Consensus Actually Fires

Judge Score Result	Action
Clearly passes all five criteria	Approve directly, no Consensus call
Clearly fails (for example, safety or accuracy critically low)	Reject directly, send back to Builder
Borderline on any criterion	Invoke Consensus Agent for that criterion only

7. Data Flow & State

7.1 Pipeline State Object

Conceptually, each project in progress is tracked as a single state record containing: an id, the user id, the classified intent type, the requirements object and whether it's complete, any outstanding clarification questions, the plan's steps, any research findings, the Builder's output artifacts, the Reviewer result, the Critic's concerns, the Judge result, the Consensus result (if invoked), a status field (clarifying, planning, building, reviewing, judging, complete, or failed), and created/updated timestamps.

7.2 Memory System

Memory Type	Scope	V1 Storage	Retention Rule
Short-term	Single conversation turn	In-memory / session	Cleared on session end
Project memory	One project across its lifecycle	Database, keyed by project id	Persists until the user deletes the project
Long-term preference	User-level, across projects	Database, keyed by user id	Persists indefinitely unless the user clears it

Hard rule carried over unchanged from all three constitutions: passwords, PINs, API keys, and other credentials are never written to any memory layer, even temporarily.

8. Agent Fallibility & Error Containment

8.1 Why This Section Exists

Every agent in this pipeline is an LLM call, and every LLM call can be wrong. Section 5 defines what each agent is supposed to do; this section defines what happens when an agent does it incorrectly — including when the agent that is supposed to catch a mistake is itself the one making it.

Three distinct failure modes are addressed:

1. **A single agent gives a wrong verdict.** For example, Reviewer marks a flawed deliverable as passing, because Reviewer is a fallible LLM call, not a guaranteed-correct checker.
2. **A wrong verdict propagates silently.** Once one agent has said “this passed,” downstream agents that see that verdict in their context may anchor on it and look less critically at the same issue, letting the original mistake travel untouched through the rest of the pipeline.
3. **Correlated failure across agents.** If Reviewer, Critic, and Judge are all the same underlying model family, they may share the same blind spot and all miss the same problem independently, which defeats the purpose of having multiple checking agents at all.

8.2 Containment Rules

Rule	What It Prevents
No single agent’s approval is final. Reviewer’s pass does not release a deliverable to the user — Judge approval is always required on top of it.	A single fallible verdict (failure mode 1) reaching the user unchecked.
If Reviewer and Critic disagree on whether an issue is real, Thinker Core automatically routes the disputed point to the Consensus Agent rather than letting Judge break the tie alone.	A disagreement getting silently resolved by whichever agent happens to run last (failure mode 2).
Critic’s prompt explicitly withholds Reviewer’s verdict until after Critic has formed its own opinion, then asks Critic to compare.	Critic anchoring on “Reviewer already said it’s fine” and skipping its own independent check (failure mode 2).
For a defined set of high-risk categories (authentication, payment handling, data deletion, permission/access control), Judge cannot approve solely because Reviewer and Critic both passed — Consensus Agent is invoked automatically regardless of score.	A single shared blind spot across same-family models approving dangerous code unchallenged (failure mode 3).
Every rejection or override (Critic overruling Reviewer, Consensus overruling Judge) is logged with both verdicts and the reasoning, not just the final outcome.	Loss of visibility into how often agents actually disagree, which is needed to tell if this system is working.

8.3 What This Does Not Solve

This containment strategy reduces the chance of a bad deliverable reaching the user undetected — it does not guarantee correctness. Consensus Agent’s majority vote can still be wrong if a mistake is common enough across providers (for example, a genuinely obscure security issue all three models fail to recognize). No automated review pipeline replaces a human inspecting high-stakes output, particularly around authentication, payments, or anything touching real user data. V1 should be treated as catching the common cases, not as a substitute for judgment on anything that matters a lot.

9. Technology Stack (V1 Recommendation)

Layer	Choice	Reasoning
Frontend	React + Vite	Matches existing developer experience
Backend	Node.js (Express) or Firebase Cloud Functions	Lightweight, matches existing Firebase familiarity
Database	Firestore	Document model fits the pipeline state object well
Auth	Firebase Auth	Reuse existing pattern
LLM Access	Direct API calls to each provider, made only from the backend, never the client	Protects API keys, matches the Resource Protection Law
Hosting	Firebase Hosting or Vercel	Simple, matches existing deployment habits

This stack choice is a recommendation, not a constitutional requirement. It is chosen because it overlaps heavily with tools already familiar, minimizing new tooling to learn for a V1 build.

10. Build Order (Implementation Roadmap)

This sequencing turns the constitutions' broader roadmap into concrete, shippable milestones.

Phase	Deliverable
Phase 1	Thinker Core skeleton plus the direct chat path (no agents yet, just passthrough chat)
Phase 2	Intent Agent and Clarification Agent, with the hard gate between them
Phase 3	Planner Agent and Builder Agent (single-step, no research yet)
Phase 4	Reviewer Agent and the one-retry loop back to Builder
Phase 5	Critic Agent and Judge Agent (Judge decides alone, no Consensus yet)
Phase 6	Consensus Agent and the borderline-routing logic from Judge
Phase 7	Research Agent (optional step augmentation)
Phase 8	Project memory persistence and long-term preference memory
Phase 9	Polish: UI, error states, retry UX, and the launch checklist (Section 11)

Each phase should be independently testable before moving to the next.

11. V1 Launch Checklist

- Stable chat experience: the direct path works without errors
- Full agent pipeline completes end-to-end on at least three different project types (app, website, game)
- Clarification gate cannot be bypassed
- No credentials ever appear in stored memory (manual audit)
- Consensus Agent tested with intentionally borderline cases
- High-risk categories (auth, payment, data deletion, permissions) tested to confirm Consensus fires automatically, even when Reviewer and Critic both pass
- Token cost per request is logged and reviewed
- Final response is always a single merged answer, never raw agent dumps

12. Explicitly Out of Scope for V1

Carried forward from the longer-term vision, not to be built now:

- Parallel multi-step Builder execution
- Automatic re-planning when a step fails repeatedly
- Native mobile app (the Play Store launch can wrap the web app initially)
- Organizational/team-shared memory
- Autonomous multi-day project system with no human checkpoint
- Native Thinker foundation model (V1 uses third-party model APIs only)

13. Open Decisions for Founder Review

These are choices this document made to keep V1 buildable. Flag any that should change before development starts:

1. Consensus uses exactly three providers (three different model vendors) — confirm provider access and budget exists for all three.
2. Reviewer retry is capped at one loop — confirm this is acceptable or should be configurable.
3. Research Agent is optional and skippable per step — confirm this matches intended behavior, rather than “always research.”
4. The technology stack assumes continuity with an existing Firebase-based project — confirm this is desired versus a fresh stack.
5. The high-risk category list for mandatory Consensus (Section 8.2) currently covers authentication, payments, data deletion, and permissions — confirm this list is complete for your use case, or add categories specific to your projects.

End of V1 Technical Architecture Specification.